# Compiler Design Assignment 2 (CO2)

Compiler Design Assignment 2 (CO2)

This document summarizes the clean restart of the original scenario-based CO2 assignment and documents the verified outputs from the five parser-design tasks.

The report is based on the original assessment paper in Question_Paper/AT2_SCENARIO_BASED_TASK.pdf and the actual execution evidence generated from the project source files.

## Q1 - LL(1) Parsing Table Conflict

Implementation source: Source_Code/Q1/ll1_conflict_demo.py

Execution output file: Q1_ll1_conflict.txt

Q1: LL(1) Parsing Table Conflict and FIRST/FOLLOW Analysis

==========

Scenario: A student constructs an LL(1) parsing table with multiple entries in one cell.

Issue: Overlapping FIRST/FOLLOW sets create a deterministic parsing conflict.

Problematic grammar:

S -> aS | a

FIRST(aS) = {a}

FIRST(a) = {a}

Therefore, M[S, a] contains two entries:

M[S, a] = { S -> aS, S -> a }

This is a FIRST/FIRST conflict, so the grammar is not LL(1).

Reason: the syntax analyzer must choose a production using one lookahead token, but the cell has two valid choices.

Result: deterministic parsing fails, and the parser cannot build a valid predictive table.

Decision: left factoring the grammar produces a valid LL(1) form.

Left-factored grammar:

S -> aS'

S' -> S | epsilon

FIRST(S) = {a}

FIRST(S') = {a, epsilon}

FOLLOW(S) = {$}

FOLLOW(S') = {$}

FIRST(S) intersect FOLLOW(S') = {a} intersect {$} = empty set

Hence the grammar satisfies LL(1) conditions.

Parsing table after repair:

Non-terminal a $

S S -> aS'

S' S' -> S S' -> epsilon

Input: aa$

S => aS'

S' => S => aS'

S' => epsilon

ACCEPTED -> deterministic LL(1) parsing succeeds.

Conclusion: overlapping FIRST/FOLLOW sets are the source of the conflict; removing left recursion, left factoring, and ambiguity resolves the issue.

## Q2 - Parse Tree Associativity

Implementation source: Source_Code/Q2/parse_tree_demo.py

Execution output file: Q2_parse_tree.txt

Q2: Incorrect Parse Tree Generation and Associativity

==========

Input: id + id - id

Ambiguous grammar:

E -> E + E | E - E | id

Right tree (incorrect):

E

/ | \

id + E

/ | \

id - id

Grouping: id + (id - id)

Evaluation: 5 + (6 - 7) = 4

Left tree (correct):

E

/ | \

E - id

/ | \

id + id

Grouping: (id + id) - id

Evaluation: (5 + 6) - 7 = 4

Issue: the grammar does not specify associativity. As a result, the same input can produce two different parse trees.

Correct design:

E -> E + T | E - T | T

T -> id

This grammar forces left associativity and gives a single parse tree.

Result: correct tree shape is ((id + id) - id), and evaluation is deterministic and accurate.

Conclusion: grammar structure determines tree shape, and associativity must be fixed explicitly to prevent wrong evaluation.

## Q3 - Error Detection and Recovery

Implementation source: Source_Code/Q3/error_handling_demo.py

Execution output file: Q3_error_handling.txt

Q3: Invalid Expression Handling and Error Recovery

==========

Input tokens: id + * id

Lexical analysis succeeds: id, +, *, id are all valid tokens.

But the token arrangement is invalid, so a syntax error occurs.

Grammar:

E -> E + T | T

T -> T * F | F

F -> (E) | id

Predictive parsing observation:

After matching `id +`, the parser expects an operand before `*`.

The grammar allows `T -> F` and `F -> id`, but `*` cannot begin an operand.

Therefore, at the lookahead `*`, the parsing table has no entry and an error is raised.

Error report:

Syntax error at "*": operand expected after "+"

Recovery techniques:

1. Panic mode: discard tokens until a synchronizing token is reached.

2. Phrase-level recovery: insert the missing operand id before *.

3. Error productions: E -> E + error

Recovery result:

Panic mode: skip "*" and continue parsing with valid expression `id + id`.

Phrase level: insert `id` before `*` to recover the valid form `id + id * id`.

Conclusion: a good parser reports the exact error, recovers quickly, and avoids cascading false errors.

## Q4 - Dangling Else Resolution

Implementation source: Source_Code/Q4/dangling_else_demo.py

Execution output file: Q4_dangling_else.txt

Q4: Dangling Else Problem and Resolution

==========

Grammar:

S -> if E then S | if E then S else S | a

Input: if E1 then if E2 then a else a

This creates two valid parse trees:

1) else belongs to the inner if

2) else belongs to the outer if

The second tree is the ambiguous one, and the grammar is not conflict-free.

The parser cannot decide which `if` the `else` should attach to with a single lookahead token.

This is the classic dangling else problem.

Nearest-then rule:

Every else attaches to the nearest unmatched then.

Under this rule, the else joins the inner if.

Recommended reconstruction:

S -> M | U

M -> if E then M else M | a

U -> if E then S | if E then M else U

This rule set removes ambiguity and ensures a single, deterministic parse tree.

Conclusion: the dangling else is a shift/reduce conflict in LR parsing and must be resolved by grammar redesign or by the nearest-then convention.

## Q5 - Operator Precedence Conflict

Implementation source: Source_Code/Q5/operator_precedence_demo.py

Execution output file: Q5_operator_precedence.txt

Q5: Operator Precedence Conflict and Grammar Redesign

==========

Ambiguous grammar:

E -> E + E | E * E | id

Input: id + id * id

Two possible groupings exist:

1. id + (id * id) -> correct because * has higher precedence than +

2. (id + id) * id -> wrong because + is evaluated before *

Issue: the grammar gives equal precedence to + and *, so a shift/reduce conflict appears.

The parser cannot decide whether to reduce or shift when it sees `*` after `E + E`.

Correct precedence-based grammar:

E -> E + T | T

T -> T * F | F

F -> id

This enforces:

1. `*` is evaluated before `+` because `T` appears one level below `E`.

2. Left associativity is preserved by left-recursive rules.

3. The parse tree is unique and evaluation is consistent.

Correct evaluation:

id + (id * id) = 5 + (6 * 7) = 47

Conclusion: precedence and associativity must be encoded in the grammar or by parser declarations so that the same expression yields one parse tree and one evaluation.

## Final Summary

All five scenario-based assignments were implemented, executed, and verified using the project output files in Output_Files/. The evidence confirms that the parser design issues were correctly identified, explained, and resolved by applying LL(1) validation, associativity fixes, recovery strategies, nearest-then rules, and precedence-based grammar redesign.